Lesson 06 Demo 02

Creating a React Application Using the Principles of Redux

Objective: To develop a React application using the redux principle to manage global state efficiently, ensure predictable state updates, and improve scalability

Tools Required: Node Terminal, React app, and Visual Studio Code

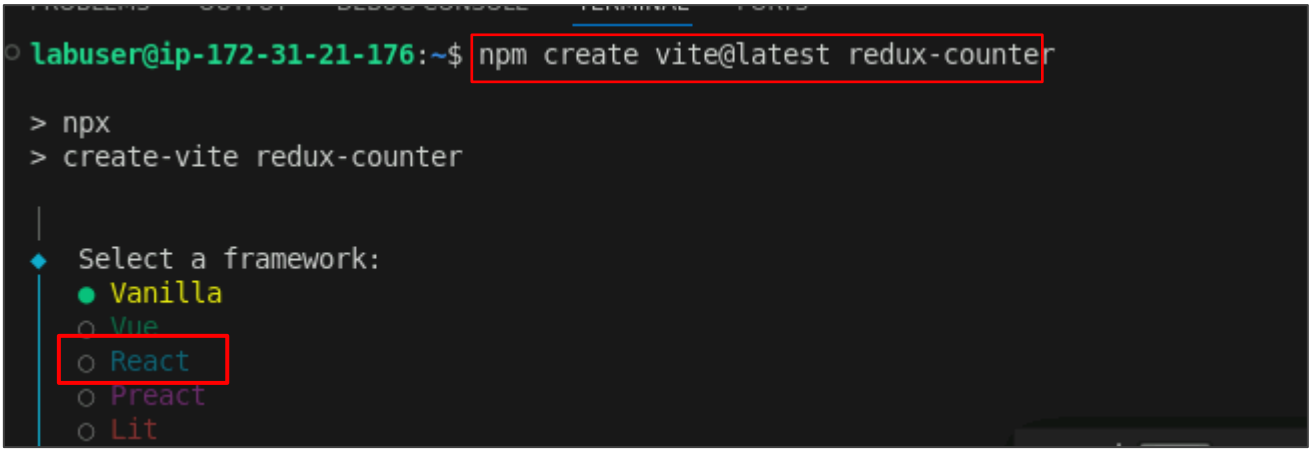
Prerequisites: Knowledge of creating a React app and understanding of the folder structure

Steps to be followed:

1. Create a new React app
2. Create a new file called store.js
3. Open the App.js file
4. Wrap the App component with the Provider component
5. Run the application

Step 1: Create a new React app

1.1 Open the terminal, execute the given command to create a new project, and select **React** as the **framework**:
npm create vite@latest redux-counter



1.2 Select **JavaScript** as a **variant**

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

◇ Select a framework:
  React

◆ Select a variant:
  ● TypeScript
  ○ TypeScript + SWC
  ○ JavaScript
  ○ JavaScript + SWC
  ○ React Router v7 ↗
```

1.3 Execute the given command to create a new React app with the name redux-counter:
cd redux-counter

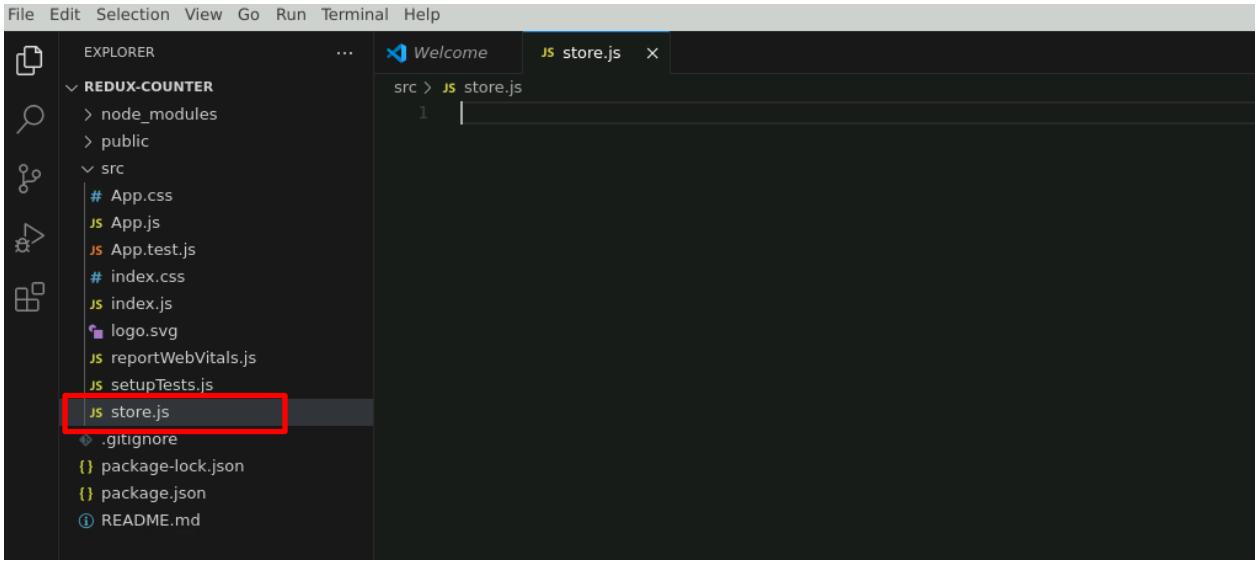
```
  cd redux-counter
  npm start

Happy hacking!
labuser@ip-172-31-45-231:~$ cd redux-counter
labuser@ip-172-31-45-231:~/redux-counter$
```

1.4 Install the necessary dependencies by running the given command:
npm install redux react-redux

```
labuser@ip-172-31-45-231:~/redux-counter$ npm install redux react-redux

added 4 packages, and audited 1347 packages in 7s

268 packages are looking for funding
  run `npm fund` for details

12 vulnerabilities (6 moderate, 6 high)

To address all issues (including breaking changes), run:
  npm audit fix --force
```

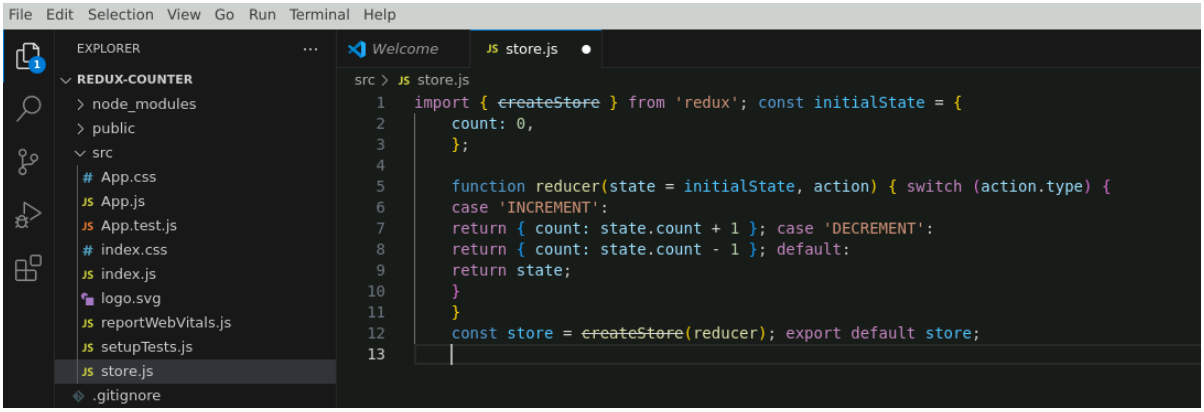
Step 2: Create a new file called store.js

2.1 Create a **store.js** file in the **src** directory



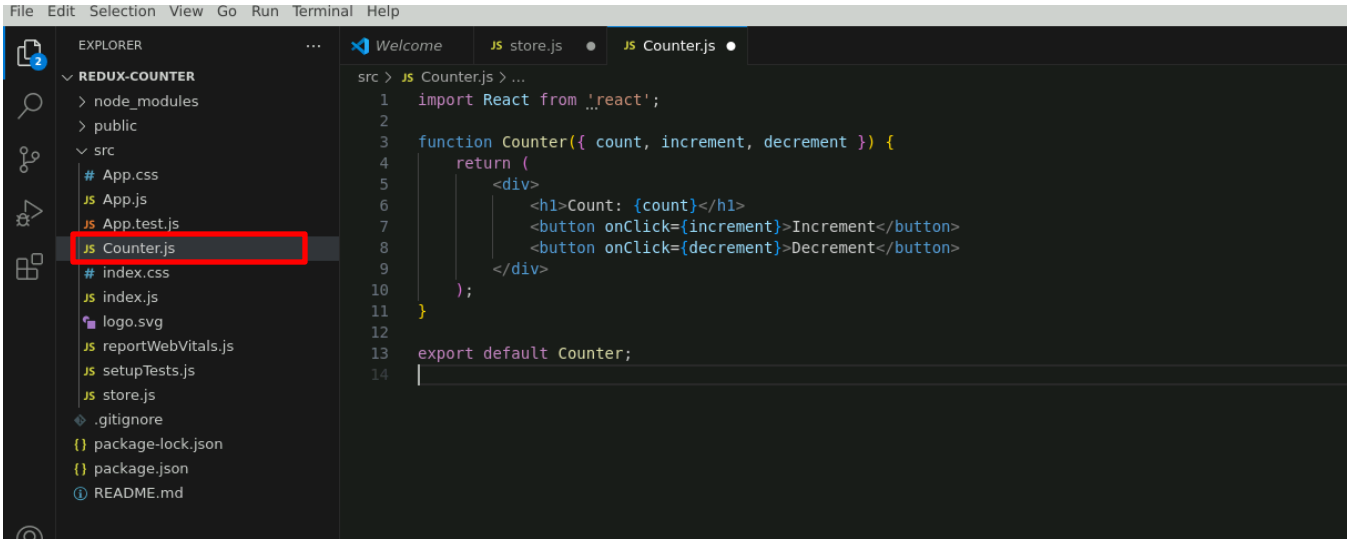
2.2 Enter the following code to the **store.js** file:

```
import { createStore } from 'redux'; const initialState = {
count: 0,
};

function reducer(state = initialState, action) { switch (action.type) {
case 'INCREMENT': return { count: state.count + 1 };
case 'DECREMENT': return { count: state.count - 1 };
default: return state;
}
}
const store =createStore(reducer); export default store;
```



2.3 Create a new file called **Counter.js** in the directory **src/components** and enter the following code inside the file:

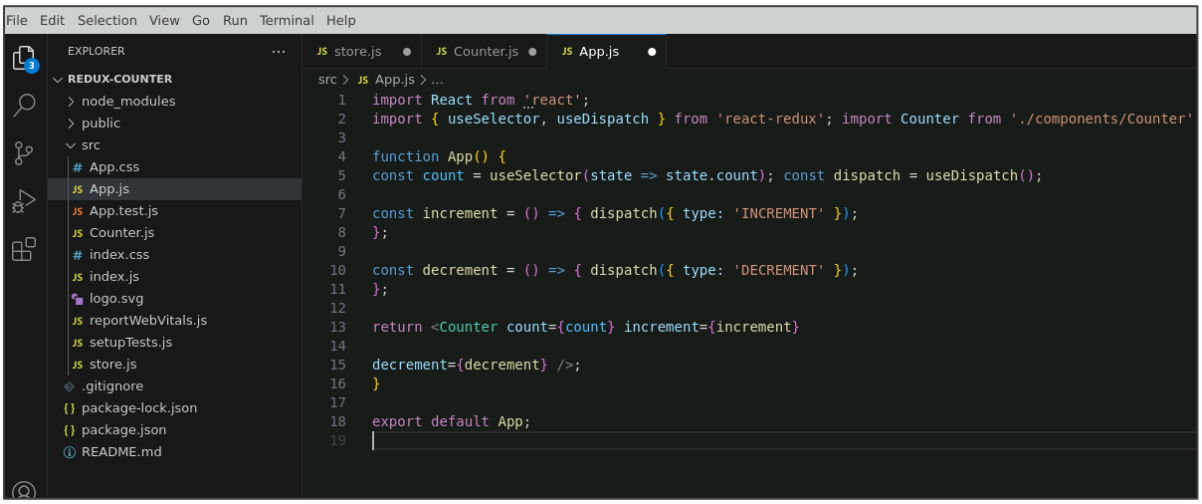
```
import React from 'react';

function Counter({ count, increment, decrement }) {
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
}

    export default Counter;
```



Step 3: Open the App.js file

3.1 In the **src** directory, open the existing file named **App.js** and enter the following code:

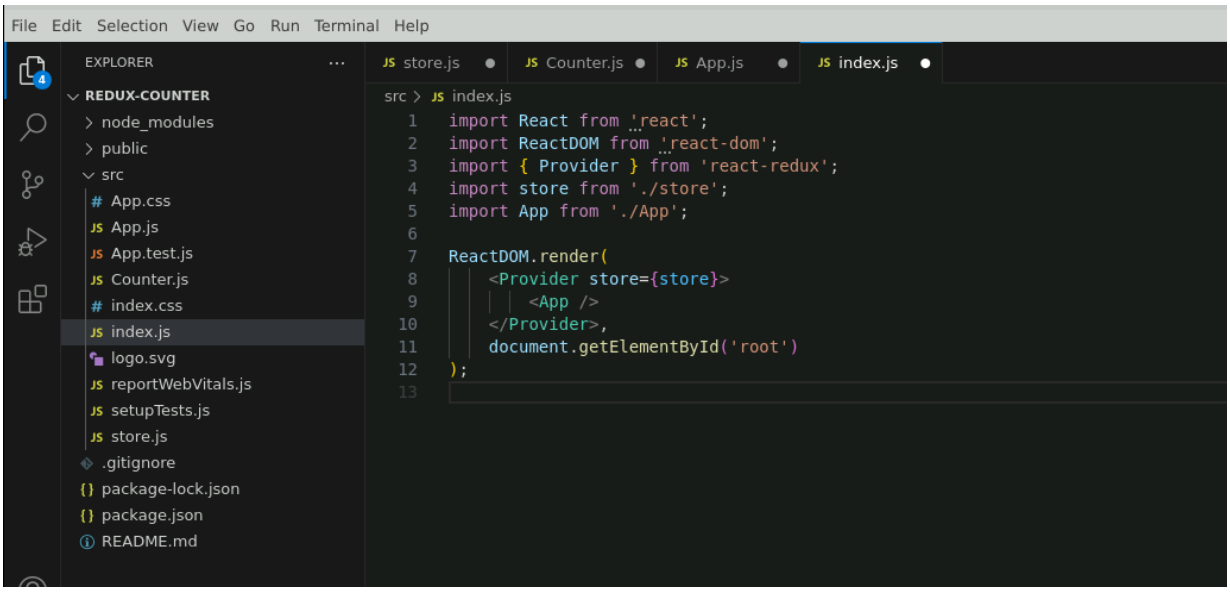
```
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import Counter from './components/Counter';

function App() {
const count = useSelector(state => state.count);
const dispatch = useDispatch();

const increment = () => {
dispatch({ type: 'INCREMENT' });
};
const decrement = () => {
dispatch({ type: 'DECREMENT' });
};
return <Counter count={count} increment={increment}
decrement={decrement} />;
}
export default App;
```



Step 4: Wrap the App component with the Provider component

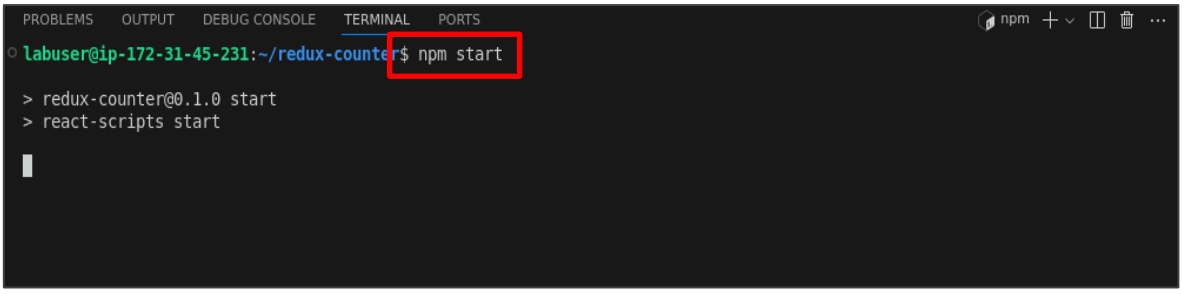
4.1 In the **src** directory, open the existing **index.js** file and enter the following code:

```
import React from 'react';
import ReactDOM from 'react-dom';
import { Provider } from 'react-redux';
import store from './store';
import App from './App';

ReactDOM.render(
  <Provider store={store}>
    <App />
  </Provider>,
  document.getElementById('root')
  );
```

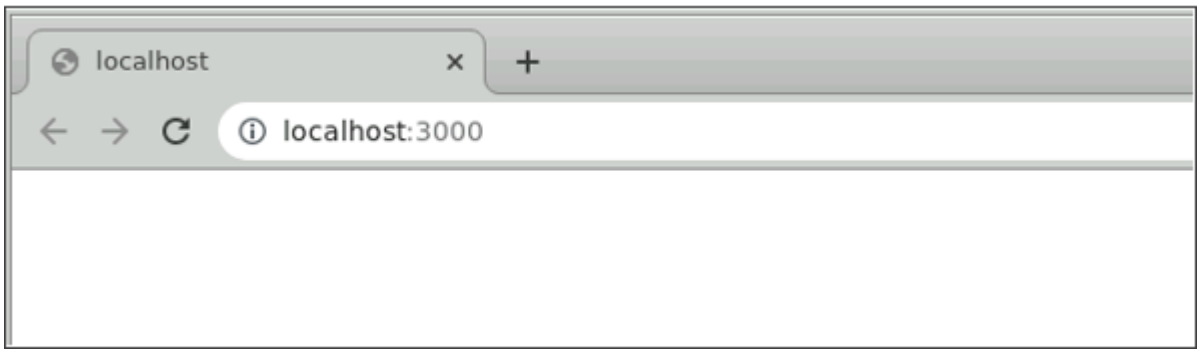


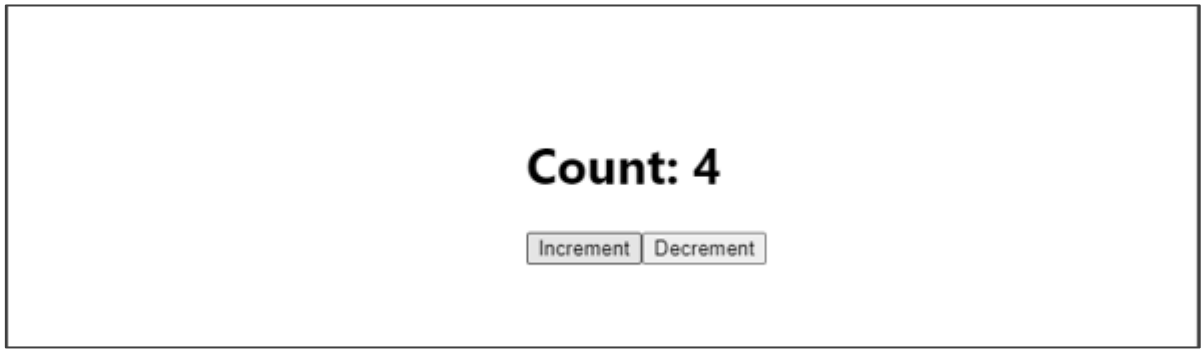
Step 5: Run the application

5.1 Run the given command to start the development server:
npm start



Once the application starts, it will redirect you to the following web page:





By following these steps, you have successfully created a React application, managing the state through a centralized store and connecting components for efficient state management.